[SD]

Scam Message Detector

Complete Developer Guide — Django + Python + Frontend

Stack	Python 3.10+ · Django 4.2 · Django REST Framework
Frontend	Vanilla HTML / CSS / JavaScript
Detection	Rule-based NLP engine with weighted risk scoring
Database	SQLite (dev) / PostgreSQL (prod)
Version	1.0 · 2025

This guide walks you through building a full-stack scam detection system from scratch. It covers project setup, the detection engine, Django models, REST API views, serializers, URL routing, and the complete frontend template — all in one place.

Table of Contents

1. Project Overview & Architecture

The Scam Message Detector is a full-stack web application that analyzes text messages for fraud indicators in real time. Users paste a message into the dashboard, the frontend POSTs it to a Django REST endpoint, the detection engine scores it against weighted pattern categories, and the result — risk level, score 0-100, and matched flags — is returned as JSON and rendered instantly in the UI.

Architecture Flow

Step	Component	Responsibility
①	Frontend	User pastes message → fetch() POST to /api/analyze/
②	Django View	Validates input with DRF serializer
③	analyzer.py	Runs rule engine → returns AnalysisResult dataclass
④	Model / DB	Saves result to ScannedMessage table (SQLite/Postgres)
⑤	JSON Response	Returns score, risk_level, flags, explanation
⑥	Frontend	Renders risk badge, score bar, flag cards, history

Risk Levels

Level	Score Range	Color	Meaning
Safe	0 – 14	Green	No scam indicators detected
Caution	15 – 39	Amber	Mild warning signs, verify sender
Suspicious	40 – 69	Orange	Multiple red flags — do not comply
Scam	70 – 100	Red	High confidence fraud attempt

2. Project Setup

Folder Structure

```
scam_detector/

■■■ manage.py

■■■ requirements.txt

■■■ scam_detector/

■ ■■■ settings.py

■ ■■■ urls.py

■ ■■■ wsgi.py

■■■ detector/

■■■ models.py

■■■ views.py

■■■ urls.py

■■■ serializers.py

■■■ analyzer.py ← core detection engine

■■■ templates/

■■■ detector/

■■■ index.html
```

requirements.txt

```
django>=4.2

djangorestframework

django-cors-headers

scikit-learn
```

```
nltk
```

Bootstrap Commands

```
# 1. Create virtual environment

python -m venv venv

source venv/bin/activate # Windows: venv\Scripts\activate


# 2. Install dependencies

pip install -r requirements.txt


# 3. Create Django project and app

django-admin startproject scam_detector .

python manage.py startapp detector


# 4. Run migrations

python manage.py makemigrations

python manage.py migrate


# 5. Create superuser (optional)

python manage.py createsuperuser


# 6. Start development server

python manage.py runserver
```

3. Django Settings (settings.py)

Add the following to your INSTALLED_APPS, MIDDLEWARE, and template settings:

```
# scam_detector/settings.py


INSTALLED_APPS = [

'django.contrib.admin',

'django.contrib.auth',

'django.contrib.contenttypes',

'django.contrib.sessions',

'django.contrib.messages',

'django.contrib.staticfiles',

# Third-party

'rest_framework',

'corsheaders',

# Local

'detector',

]


MIDDLEWARE = [

'corsheaders.middleware.CorsMiddleware', # must be first

'django.middleware.security.SecurityMiddleware',

# ... rest of default middleware

]


CORS_ALLOW_ALL_ORIGINS = True # tighten in production
```

```
TEMPLATES = [{

'BACKEND': 'django.template.backends.django.DjangoTemplates',

'DIRS': [],

'APP_DIRS': True, # enables app-level templates/

...

}]
```

Production note	In production set CORS_ALLOW_ALL_ORIGINS = False and whitelist only your frontend domain with CORS_ALLOWED_ORIGINS = ['https://yourdomain.com'].

4. Detection Engine (detector/analyzer.py)

This is the brain of the system. It uses a weighted pattern dictionary across five threat categories. Each category has a weight (how much it adds to the risk score) and a list of regex patterns. Safe signal patterns reduce the score.

Pattern Categories & Weights

Category	Weight	What it looks for
Urgency Pressure	25	act now, expires today, urgent, limited time
Financial Lure	30	you've won, claim prize, send KSH/USD, M-PESA, bitcoin
Impersonation	25	Safaricom, KRA, your bank, verify your account
Personal Info Harvest	20	National ID, PIN, OTP, click this link, login here
Too Good To Be True	30	lottery, inheritance, million, congratulations

Full Code — analyzer.py

```
import re

from dataclasses import dataclass, field


SCAM_PATTERNS = {

"urgency_pressure": {

"weight": 25,

"patterns": [

r"act now", r"limited time", r"expires today",

r"urgent", r"immediately", r"don't delay", r"last chance"

]

},

"financial_lure": {

"weight": 30,
```

```
"patterns": [
r"you('ve| have) won", r"claim your (prize|reward|money)",
r"free money", r"send (ksh|usd|kes|money)",
r"mpesa", r"wire transfer", r"bitcoin", r"crypto wallet",
r"bank details", r"account number"
]
},
"impersonation": {
"weight": 25,
"patterns": [
r"safaricom", r"kenya revenue authority", r"kra",
r"your bank", r"paypal security", r"verify your account",
r"confirm your (pin|password|details)"
]
},
"personal_info_harvest": {
"weight": 20,
"patterns": [
r"id number", r"national id", r"send (your|ur) (pin|password)",
r"otp", r"verification code", r"share the code",
r"click this link", r"login here"
]
},
"too_good_to_be_true": {
```

```
"weight": 30,

"patterns": [

r"\d{4,} (ksh|dollars|usd|kes)", r"million",

r"lottery", r"inheritance", r"you are selected",

r"congratulations"

]

},

}


SAFE_SIGNALS = [

r"invoice attached", r"meeting at", r"please find",

r"regards", r"sincerely",

r"thank you for your (order|purchase)"

]


@dataclass

class AnalysisResult:

score: int = 0

risk_level: str = "safe"

flags: list = field(default_factory=list)

explanation: str = ""

is_scam: bool = False


def analyze_message(text: str) -> AnalysisResult:

result = AnalysisResult()
```

```
text_lower = text.lower()

safe_hits = sum(1 for p in SAFE_SIGNALS if re.search(p, text_lower))

score = max(0, -10 * safe_hits)


for category, data in SCAM_PATTERNS.items():

hits = [p for p in data['patterns'] if re.search(p, text_lower)]

if hits:

score += data['weight'] * len(hits)

result.flags.append({

"category": category.replace("_", " ").title(),

"matched": hits[:3],

"severity": "high" if data["weight"] >= 25 else "medium"

})


result.score = min(score, 100)


if result.score >= 70:

result.risk_level = "scam"

result.is_scam = True

result.explanation = "High probability scam. Do not respond or send any money."

elif result.score >= 40:

result.risk_level = "suspicious"

result.explanation = "Suspicious. Verify through official channels."

elif result.score >= 15:

result.risk_level = "caution"
```

```
result.explanation = "Mild warning signs. Proceed carefully."

else:

result.risk_level = "safe"

result.explanation = "No obvious scam patterns detected."


return result
```

5. Database Model (detector/models.py)

Each analyzed message is persisted in the ScannedMessage table.

```
from django.db import models


class ScannedMessage(models.Model):

RISK_CHOICES = [

('safe', 'Safe'),

('caution', 'Caution'),

('suspicious', 'Suspicious'),

('scam', 'Scam'),

]

text = models.TextField()

score = models.IntegerField(default=0)

risk_level = models.CharField(max_length=20, choices=RISK_CHOICES)

is_scam = models.BooleanField(default=False)

flags = models.JSONField(default=list)

explanation = models.TextField(blank=True)

created_at = models.DateTimeField(auto_now_add=True)


class Meta:

ordering = ['-created_at']


def __str__(self):

return f'[{self.risk_level.upper()}] {self.text[:60]}...'
```

Migration	After adding this model run: python manage.py makemigrations && python manage.py migrate

6. Serializers (detector/serializers.py)

```
from rest_framework import serializers

from .models import ScannedMessage


class MessageInputSerializer(serializers.Serializer):

text = serializers.CharField(min_length=5, max_length=5000)


class ScannedMessageSerializer(serializers.ModelSerializer):

class Meta:

model = ScannedMessage

fields = '__all__'
```

7. API Views (detector/views.py)

```
from rest_framework.views import APIView

from rest_framework.response import Response

from rest_framework import status

from django.shortcuts import render

from .analyzer import analyze_message

from .models import ScannedMessage

from .serializers import MessageInputSerializer, ScannedMessageSerializer


class AnalyzeMessageView(APIView):
```

```
def post(self, request):

serializer = MessageInputSerializer(data=request.data)

if not serializer.is_valid():

return Response(serializer.errors,

status=status.HTTP_400_BAD_REQUEST)

text = serializer.validated_data['text']

result = analyze_message(text)

record = ScannedMessage.objects.create(

text=text, score=result.score,

risk_level=result.risk_level, is_scam=result.is_scam,

flags=result.flags, explanation=result.explanation,

)

return Response(ScannedMessageSerializer(record).data,

status=status.HTTP_201_CREATED)


class RecentScansView(APIView):

def get(self, request):

scans = ScannedMessage.objects.all()[:20]

return Response(ScannedMessageSerializer(scans, many=True).data)


def dashboard(request):

return render(request, 'detector/index.html')
```

8. URL Routing

detector/urls.py

```
from django.urls import path

from . import views


urlpatterns = [

path('', views.dashboard, name='dashboard'),

path('api/analyze/', views.AnalyzeMessageView.as_view()),

path('api/history/', views.RecentScansView.as_view()),

]
```

scam_detector/urls.py (root)

```
from django.contrib import admin

from django.urls import path, include


urlpatterns = [

path('admin/', admin.site.urls),

path('', include('detector.urls')),

]
```

API Endpoints Summary

Method	URL	Description
GET	/	Renders the frontend dashboard
POST	/api/analyze/	Analyzes a message, returns risk result
GET	/api/history/	Returns last 20 scanned messages
GET	/admin/	Django admin panel

9. Frontend Template (detector/templates/detector/index.html)

Save this file at detector/templates/detector/index.html. It is a single self-contained HTML file with embedded CSS and JavaScript. The JavaScript fetch() call POSTs to /api/analyze/ and renders the result dynamically.

HTML Structure Overview

Section	Element / ID	Purpose
Navbar	<nav>	Brand logo, beta badge, admin link
Stats bar	.stats-grid	4 counters: total, scams, suspicious, safe
Input panel	#msg-input	Textarea + Analyze button + sample pills
Result box	#result-box	Risk icon, label, score bar, flag cards
History panel	#history-list	Last 10 scanned messages with pills

Complete index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8"/>

<meta name="viewport" content="width=device-width, initial-scale=1.0"/>

<title>Scam Message Detector</title>

<style>

* { box-sizing: border-box; margin: 0; padding: 0; }

body {

font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;

background: #f5f4f0; color: #1a1a1a; min-height: 100vh;

}
```

```
.navbar {
background: #fff; border-bottom: 1px solid #e5e3dc;
padding: 0 2rem; height: 56px;
display: flex; align-items: center; justify-content: space-between;
}
.nav-brand { font-size:15px; font-weight:500;
display:flex; align-items:center; gap:8px; }
.nav-dot { width:10px; height:10px; border-radius:50%; background:#A32D2D; }
.nav-badge { font-size:10px; background:#FCEBEB; color:#A32D2D;
padding:2px 7px; border-radius:4px; font-weight:500; }
.container { max-width:900px; margin:0 auto; padding:2rem 1.5rem; }
.page-title { font-size:22px; font-weight:500; margin-bottom:4px; }
.page-sub { font-size:14px; color:#666; margin-bottom:2rem; }
.stats-grid { display:grid; grid-template-columns:repeat(4,1fr);
gap:12px; margin-bottom:1.5rem; }
@media (max-width:600px) { .stats-grid { grid-template-columns:repeat(2,1fr); } }
.stat-card { background:#fff; border:1px solid #e5e3dc;
border-radius:10px; padding:14px 16px; }
.stat-label { font-size:12px; color:#888; margin-bottom:4px; }
.stat-value { font-size:26px; font-weight:500; }
.stat-value.red { color:#A32D2D; }
.stat-value.amber { color:#854F0B; }
.stat-value.green { color:#3B6D11; }
.panel { background:#fff; border:1px solid #e5e3dc;
```

```
border-radius:12px; padding:1.5rem; margin-bottom:1.25rem; }

.panel-title { font-size:11px; font-weight:500; text-transform:uppercase;

letter-spacing:.05em; color:#888; margin-bottom:14px; }

textarea {

width:100%; border:1px solid #d3d1c7; border-radius:8px;

padding:12px 14px; font-size:14px; font-family:inherit;

resize:vertical; min-height:130px; line-height:1.6;

color:#1a1a1a; background:#fafaf8;

}

textarea:focus { outline:none; border-color:#7F77DD;

box-shadow:0 0 0 3px rgba(127,119,221,.1); }

.btn-row { display:flex; gap:10px; margin-top:12px; }

button {

cursor:pointer; border:1px solid #d3d1c7; border-radius:8px;

padding:9px 20px; font-size:13px; font-weight:500;

background:transparent; transition:background .15s;

}

button:hover { background:#f5f4f0; }

button.primary { background:#A32D2D; border-color:#A32D2D; color:#fff; }

button.primary:hover { background:#791F1F; }

.samples { display:flex; flex-wrap:wrap; gap:6px; margin-top:10px; }

.sample-btn {

font-size:11px; padding:4px 12px; border-radius:20px;

background:#f5f4f0; border:1px solid #d3d1c7;
```

```
cursor:pointer; color:#666;
}
.sample-btn:hover { border-color:#888; color:#1a1a1a; }
.loading { display:none; align-items:center; gap:8px;
font-size:13px; color:#888; margin-top:12px; }
.spinner { width:16px; height:16px; border:2px solid #d3d1c7;
border-top-color:#888; border-radius:50%;
animation:spin .7s linear infinite; }
@keyframes spin { to { transform:rotate(360deg); } }
.result-box { display:none; border-radius:10px; padding:1.25rem;
margin-top:1rem; border:1.5px solid; }
.result-box.safe { background:#EAF3DE; border-color:#639922; color:#27500A; }
.result-box.caution { background:#FAEEDA; border-color:#BA7517; color:#633806; }
.result-box.suspicious { background:#FAEEDA; border-color:#E24B4A; color:#411A02; }
.result-box.scam { background:#FCEBEB; border-color:#E24B4A; color:#501313; }
.risk-header { display:flex; align-items:center; gap:14px; margin-bottom:14px; }
.risk-icon {
width:40px; height:40px; border-radius:50%;
display:flex; align-items:center; justify-content:center;
font-size:16px; font-weight:700; color:#fff;
}
.risk-icon.safe { background:#639922; }
.risk-icon.caution { background:#BA7517; }
.risk-icon.suspicious { background:#E24B4A; }
```

```
.risk-icon.scam { background:#A32D2D; }

.score-meter { margin:10px 0 14px; }

.score-track { height:8px; border-radius:4px;

background:rgba(0,0,0,.1); overflow:hidden; margin-top:6px; }

.score-fill { height:100%; border-radius:4px; transition:width .7s ease; }

.fill-safe { background:#639922; }

.fill-caution { background:#BA7517; }

.fill-suspicious { background:#E24B4A; }

.fill-scam { background:#A32D2D; }

.flag-item { background:rgba(0,0,0,.06); border-radius:6px;

padding:8px 12px; margin-bottom:6px; font-size:12.5px; }

.flag-cat { font-weight:500; margin-bottom:3px; }

.flag-matches { opacity:.7; font-family:monospace; font-size:11px; }

.history-item { display:flex; align-items:center; gap:10px;

padding:9px 0; border-bottom:1px solid #e5e3dc; }

.history-item:last-child { border-bottom:none; }

.risk-dot { width:8px; height:8px; border-radius:50%; flex-shrink:0; }

.dot-safe { background:#639922; }

.dot-caution { background:#BA7517; }

.dot-suspicious { background:#E24B4A; }

.dot-scam { background:#A32D2D; }

.history-text { font-size:13px; flex:1;

white-space:nowrap; overflow:hidden; text-overflow:ellipsis; }

.history-score { font-size:12px; color:#888; flex-shrink:0; }
```

```
.pill { display:inline-block; font-size:10px;

padding:2px 7px; border-radius:10px;

font-weight:500; margin-left:6px; }

.pill-safe { background:#C0DD97; color:#27500A; }

.pill-caution { background:#FAC775; color:#633806; }

.pill-suspicious { background:#F09595; color:#501313; }

.pill-scam { background:#F7C1C1; color:#501313; }

</style>

</head>

<body>


<!-- NAVBAR -->

<nav class="navbar">

<div class="nav-brand">

<div class="nav-dot"></div>

Scam Detector

<span class="nav-badge">Beta</span>

</div>

<a href="/admin/" style="font-size:13px;color:#888;text-decoration:none;">Admin</a>

</nav>


<!-- MAIN CONTENT -->

<div class="container">

<h1 class="page-title">Scam Message Detector</h1>

<p class="page-sub">Paste any suspicious message to detect fraud or phishing.</p>
```

```
<!-- Stats Grid -->
<div class="stats-grid">
<div class="stat-card">
<div class="stat-label">Messages scanned</div>
<div class="stat-value" id="total-count">0</div>
</div>
<div class="stat-card">
<div class="stat-label">Scams detected</div>
<div class="stat-value red" id="scam-count">0</div>
</div>
<div class="stat-card">
<div class="stat-label">Suspicious</div>
<div class="stat-value amber" id="sus-count">0</div>
</div>
<div class="stat-card">
<div class="stat-label">Clean messages</div>
<div class="stat-value green" id="safe-count">0</div>
</div>
</div>

<!-- Analyze Panel -->
<div class="panel">
<div class="panel-title">Analyze a message</div>
<textarea id="msg-input"
```

```
placeholder="Paste the message here..."></textarea>

<div class="btn-row">

<button class="primary" onclick="analyzeMessage()">Analyze message</button>

<button onclick="clearAll()">Clear</button>

</div>

<div class="loading" id="loading">

<div class="spinner"></div> Analyzing patterns...

</div>

<div style="margin-top:14px">

<span style="font-size:11px;color:#888">Try a sample:</span>

<div class="samples">

<span class="sample-btn" onclick="loadSample('scam1')">Lottery win</span>

<span class="sample-btn" onclick="loadSample('scam2')">M-PESA phish</span>

<span class="sample-btn" onclick="loadSample('scam3')">Bank impersonation</span>

<span class="sample-btn" onclick="loadSample('safe1')">Legitimate invoice</span>

<span class="sample-btn" onclick="loadSample('safe2')">Meeting invite</span>

</div>

</div>

<!-- Result box -->

<div class="result-box" id="result-box">

<div class="risk-header">

<div class="risk-icon" id="risk-icon"></div>

<div>

<div style="font-size:16px;font-weight:500" id="risk-label"></div>
```

```
<div style="font-size:13px;opacity:.8;margin-top:2px" id="risk-explanation"></div>

</div>

</div>

<div class="score-meter">

<div style="font-size:12px;font-weight:500">

Risk score: <span id="score-val"></span>/100

</div>

<div class="score-track">

<div class="score-fill" id="score-fill" style="width:0%"></div>

</div>

</div>

<div id="flags-list"></div>

</div>

</div>


<!-- History Panel -->

<div class="panel">

<div class="panel-title">Recent scans</div>

<div id="history-list">

<p style="font-size:13px;color:#888">No messages analyzed yet.</p>

</div>

</div>

</div><!-- /container -->


<!-- JAVASCRIPT -->
```

```
<script>

const samples = {

scam1: 'Congratulations! You have been selected as LUCKY WINNER of KSH 50,000

in the Safaricom Annual Lottery. Send your National ID and M-PESA PIN

to 0712345678. ACT NOW — offer expires today!',

scam2: 'URGENT: Your M-PESA account has been suspended. Verify your account by

clicking this link: http://mpesa-verify.net and confirm your PIN and OTP.',

scam3: 'Dear Customer, your bank account has been flagged. To avoid suspension,

send your account number and password to our security team immediately.',

safe1: 'Hi James, please find attached the invoice for web development services

in March. Payment is due within 30 days. Regards, Alice.',

safe2: 'Hi team, meeting at 10am tomorrow to review Q1 results.

Please find the agenda attached. Sincerely, Mark.'

};


function loadSample(key) {

document.getElementById('msg-input').value = samples[key];

}


async function analyzeMessage() {

const text = document.getElementById('msg-input').value.trim();

if (!text || text.length < 5) return;

document.getElementById('loading').style.display = 'flex';

document.getElementById('result-box').style.display = 'none';

try {
```

```
const res = await fetch('/api/analyze/', {

method: 'POST',

headers: {

'Content-Type': 'application/json',

'X-CSRFToken': getCookie('csrftoken')

},

body: JSON.stringify({ text })

});

const data = await res.json();

showResult(data);

addToHistory(data);

updateStats(data.risk_level);

} catch(e) {

alert('Error connecting to API. Is Django running?');

} finally {

document.getElementById('loading').style.display = 'none';

}

}


function showResult(data) {

const box = document.getElementById('result-box');

const level = data.risk_level;

box.className = 'result-box ' + level;

box.style.display = 'block';
```

```
const icons = { safe:'checkmark', caution:'!', suspicious:'!!', scam:'X' };

const labels = { safe:'Safe', caution:'Use Caution',

suspicious:'Suspicious', scam:'Likely Scam' };

const icon = document.getElementById('risk-icon');

icon.className = 'risk-icon ' + level;

icon.textContent = icons[level];

document.getElementById('risk-label').textContent = labels[level];

document.getElementById('risk-explanation').textContent = data.explanation;

document.getElementById('score-val').textContent = data.score;

const fill = document.getElementById('score-fill');

fill.style.width = '0%';

fill.className = 'score-fill fill-' + level;

setTimeout(() => fill.style.width = data.score + '%', 50);

const fl = document.getElementById('flags-list');

if (!data.flags || data.flags.length === 0) {

fl.innerHTML = '<p style="font-size:12px;opacity:.7">No suspicious patterns
matched.</p>';

return;

}

fl.innerHTML = data.flags.map(f => `

<div class="flag-item">

<div class="flag-cat">${f.category}

<span style="font-size:10px;opacity:.6">(${f.severity})</span>

</div>
```

```
<div class="flag-matches">matched: ${f.matched.join(' . ')}</div>

</div>`).join('');

}


let history = [];

let stats = { total:0, scam:0, suspicious:0, safe:0 };


function addToHistory(data) {

history.unshift(data);

if (history.length > 10) history.pop();

document.getElementById('history-list').innerHTML = history.map(h => `

<div class="history-item">

<div class="risk-dot dot-${h.risk_level}"></div>

<div class="history-text">${h.text.substring(0,90)}...</div>

<div class="history-score">${h.score}/100

<span class="pill pill-${h.risk_level}">${h.risk_level}</span>

</div>

</div>`).join('');

}


function updateStats(level) {

stats.total++;

if (level === 'scam') stats.scam++;

else if (level === 'suspicious') stats.suspicious++;

else stats.safe++;
```

```
document.getElementById('total-count').textContent = stats.total;

document.getElementById('scam-count').textContent = stats.scam;

document.getElementById('sus-count').textContent = stats.suspicious;

document.getElementById('safe-count').textContent = stats.safe;

}


function clearAll() {

document.getElementById('msg-input').value = '';

document.getElementById('result-box').style.display = 'none';

}


function getCookie(name) {

const match = document.cookie.match(

new RegExp('(^|;\\s*)(' + name + ')=([^;]*)'));

return match ? decodeURIComponent(match[3]) : '';

}

</script>

</body>

</html>
```

10. Running the Project

```
# Apply migrations

python manage.py makemigrations

python manage.py migrate


# Start server

python manage.py runserver


# Open browser

# http://127.0.0.1:8000 → Dashboard

# http://127.0.0.1:8000/admin/ → Admin panel

# http://127.0.0.1:8000/api/analyze/ → REST API
```

Quick test via curl	curl -X POST http://127.0.0.1:8000/api/analyze/ \ -H "Content-Type: application/json" \ -d '{"text": "Congratulations! You won KSH 50000. Send your MPESA PIN now!"}'

11. API Reference

POST /api/analyze/

Request body (JSON):

```
{ "text": "Your message to analyze here" }
```

Response body (JSON):

```
{

"id": 1,

"text": "Your message...",

"score": 85,
```

```
"risk_level": "scam",

"is_scam": true,

"explanation": "High probability scam. Do not respond.",

"flags": [

{

"category": "Financial Lure",

"matched": ["mpesa", "send (ksh|usd|kes|money)"],

"severity": "high"

}

],

"created_at": "2025-06-01T10:23:45.123Z"

}
```

12. Scam Pattern Reference

The table below lists every regex pattern in the detection engine, its category, and its individual contribution to the risk score.

Category	Weight	Patterns
Urgency Pressure	25	• act now • limited time • expires today • urgent • immediately • don't delay • last chance
Financial Lure	30	• you've won • claim your prize/reward • free money • send ksh/usd/kes • mpesa • wire transfer • bitcoin • crypto wallet • bank details • account number
Impersonation	25	• safaricom • kenya revenue authority • kra • your bank • paypal security • verify your account • confirm your pin/password
Personal Info Harvest	20	• id number • national id • send your pin/password • otp • verification code • share the code • click this link • login here
Too Good To Be True	30	• large number + currency • million • lottery • inheritance • you are selected • congratulations

Extending patterns To add new patterns, open detector/analyzer.py and append regex strings to the relevant category's 'patterns' list. Increase the weight to make that category more influential in the final score.